

Core SDK Android & iOS

Core SDK reutilizabil pentru Android și iOS

Cum extragi funcționalitatea (nu UI-ul) dintr-un proiect și o integrezi în orice aplicație nativă

Autor: Echipa SmartID **Data:** 13 iulie 2026 **Versiune:** 1.0

Cuprins

1. Contextul problemei
 2. Concluzia pe scurt
 3. De ce nu se „extrage” logica din React Native
 4. Arhitectura corectă: SDK-ul ca sursă de adevăr
 5. Variante de implementare
 - 5.1 Kotlin Multiplatform (recomandat)
 - 5.2 Native per platformă
 - 5.3 Backend / API
 - 5.4 Core partajat C++ / Rust
 6. Cum se leagă React Native de SDK
 7. Tabel comparativ
 8. Cum proiectezi API-ul public al SDK-ului
 9. Împachetare și distribuție
 10. Versionare, compatibilitate și mentenanță
 11. Securitate
 12. Checklist final
-

1. Contextul problemei

Scenariul tipic: ai (sau vrei să faci) o aplicație — de exemplu în React Native — și vrei să **reutilizezi funcționalitatea** (logica de business, nu ecranele) în alte aplicații native, atât pe **Android**, cât și pe **iOS**.

Obiectivul concret: **un „core SDK”** — un pachet cu logica ta — pe care să-l integrezi ca dependență în orice proiect native, indiferent de tehnologia UI.

Acest document explică:

- de ce logica nu se poate „extrage” pur și simplu dintr-un proiect React Native;
- care e arhitectura corectă;
- ce variante tehnice ai, cu avantaje și dezavantaje;
- exemple concrete de cod;
- cum împachetezi, versionezi și distribui SDK-ul.

2. Concluzia pe scurt

Un core SDK este arhitectura corectă. Doar că nu-l „extragi” din codul JavaScript al aplicației React Native — îl **construiești separat**, ca artefact nativ, iar aplicația React Native devine doar **unul dintre consumatori**, exact ca celelalte aplicații native.

Recomandarea principală, în funcție de natura logicii:

Dacă logica ta...	Folosește
poate rula pe server (validări, reguli de business, procesări)	Backend / API
trebuie să ruleze on-device, offline, partajată nativ	Kotlin Multiplatform (KMP)
e deja scrisă ca native module per platformă	scoate-o ca .aar + CocoaPod/Swift Package
e algoritm greu, performanță maximă, cross-platform	Core C++ / Rust cu bindings

3. De ce nu se „extrage” logica din React Native

React Native **nu produce cod nativ reutilizabil** (Swift/Kotlin). Aplicația rulează **JavaScript** peste un motor JS (Hermes) și un „bridge”/JSI care controlează componente native.

Consecințe:

- Logica scrisă în JS/TypeScript **nu rulează nativ** într-o aplicație Swift sau Kotlin.
- Nu există un buton „export as native SDK” care să transforme JS-ul în librărie Android + iOS.
- A încorpora tot motorul JS (Hermes) într-o aplicație nativă doar ca să rulezi niște logică este **impractic** pentru un SDK (dimensiune, complexitate, timp de pornire).

Deci fluxul NU este „scriu în RN → extrag SDK”. **Fluxul corect este invers:** construiești SDK-ul separat, iar RN îl consumă.

GREȘIT

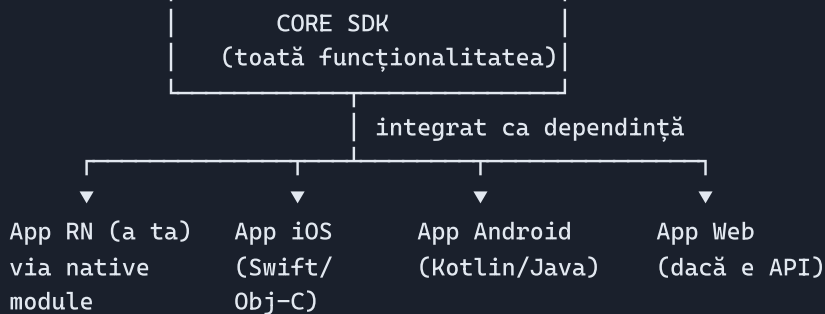
App RN (JS)
|
▼
„extrag SDK" X
(nu se poate)

CORECT

Core SDK (nativ/KMP)
|
e consumat de toți
|
▼
App RN App iOS App Android Web

4. Arhitectura corectă: SDK-ul ca sursă de adevăr

Logica reală trăiește **într-un singur loc** — SDK-ul. Toate aplicațiile îl consumă.



Beneficii:

- **O singură implementare** a logicii → o singură sursă de bug-uri și de corecturi.
- **Consistență**: toate aplicațiile se comportă identic.
- **Testare centralizată**: testezi SDK-ul o dată, temeinic.
- **Decuplare**: echipele de UI (RN, native) nu depind de detaliile interne ale logicii.

5. Variante de implementare

5.1 Kotlin Multiplatform (KMP) — recomandat

Scrii logica **o singură dată în Kotlin**, iar KMP o compilează în artefacte native pentru fiecare platformă:

- **Android** → `.aar` / artefact Maven (folosit direct în orice proiect Android)
- **iOS** → `XCFramework` (folosit din Swift/Objective-C ca orice SDK nativ)

Aceasta este soluția modernă pentru „vreau logică (nu UI) partajată nativ pe Android + iOS”. E folosită în producție de companii mari (Netflix, McDonald's, Philips, Cash App etc.).

Exemplu — cod partajat (`commonMain`):

```
// shared/src/commonMain/kotlin/com/smartid/sdk/DocumentValidator.kt
package com.smartid.sdk

data class ValidationResult(
    val isValid: Boolean,
    val errors: List<String>
)

class DocumentValidator {
    fun validateCnp(cnp: String): ValidationResult {
        val errors = mutableListOf<String>()
        if (cnp.length != 13) errors.add("CNP trebuie să aibă 13 cifre")
        if (!cnp.all { it.isDigit() }) errors.add("CNP conține caractere invalide")
        // ... restul regulilor (cifra de control etc.)
        return ValidationResult(errors.isEmpty(), errors)
    }
}
```

Cod specific platformei (`expect` / `actual`) — pentru lucruri ce diferă (criptare, storage securizat):

```
// commonMain - declarația comună
expect class SecureStorage() {
    fun save(key: String, value: String)
    fun load(key: String): String?
}
```

```
// androidMain - implementarea Android (EncryptedSharedPreferences)
actual class SecureStorage {
    actual fun save(key: String, value: String) { /* Android Keystore */ }
    actual fun load(key: String): String? { /* ... */ }
}
```

```
// iosMain - implementarea iOS (Keychain)
actual class SecureStorage {
    actual fun save(key: String, value: String) { /* Keychain */ }
    actual fun load(key: String): String? { /* ... */ }
}
```

Consum din Swift (iOS):

```
import SmartIdSDK // XCFramework generat de KMP

let validator = DocumentValidator()
let result = validator.validateCnp(cnp: "1960712123456")
if result.isValid {
    print("CNP valid")
} else {
    print("Erori: \(result.errors)")
}
```

Consum din Kotlin (Android):

```
import com.smartid.sdk.DocumentValidator

val validator = DocumentValidator()
val result = validator.validateCnp("1960712123456")
```

Avantaje:

- ✓ Un singur cod pentru logică → mentenanță minimă.
- ✓ Artefacte **native reale** (`.aar` , `XCFramework`) integrabile în orice proiect.
- ✓ Funcționează **offline**, on-device.
- ✓ Kotlin e limbaj matur, tipizat, cu tooling excelent (IntelliJ/Android Studio).
- ✓ Poți partaja și networking, serializare, bază de date (Ktor, kotlinx.serialization, SQLDelight).

Dezavantaje:

- ✗ Trebuie să (re)scrii logica în Kotlin dacă acum e în JS.
- ✗ API-ul generat pentru iOS moștenește idiomuri Kotlin (nume, nullability) — uneori mai puțin „swifty”.
- ✗ Debugging-ul pe partea iOS a codului Kotlin e mai puțin comod decât nativ pur.
- ✗ Aduagă un pas de build (Gradle) în pipeline-ul iOS.

5.2 Native per platformă

Scrii SDK-ul de **două ori**, câte o dată pentru fiecare platformă:

- **iOS** → Swift Package sau CocoaPod
- **Android** → Android Library (`.aar`)

Exemplu iOS (Swift Package):

```
// Sources/SmartIdSDK/DocumentValidator.swift
public struct ValidationResult {
    public let isValid: Bool
    public let errors: [String]
}

public final class DocumentValidator {
    public init() {}
    public func validateCnp(_ cnp: String) → ValidationResult {
        var errors: [String] = []
        if cnp.count ≠ 13 { errors.append("CNP trebuie să aibă 13 cifre") }
        // ...
        return ValidationResult(isValid: errors.isEmpty, errors: errors)
    }
}
```

Exemplu Android (Kotlin library):

```
// smartid-sdk/src/main/java/com/smartid/sdk/DocumentValidator.kt
class DocumentValidator {
    fun validateCnp(cnp: String): ValidationResult { /* ... */ }
}
```

Avantaje:

-  Cod 100% nativ, idiomatic pe fiecare platformă (cel mai „curat” pentru consumatori).
-  Control total pe performanță și pe API.
-  Debugging nativ pur.

Dezavantaje:

-  **Dublă mentenanță:** două implementări de sincronizat, dublu efort la fiecare feature/bug.
-  Risc de **divergență:** logica poate să difere subtil între platforme.
-  Testare dublă.

5.3 Backend / API

Muți logica pe un **server** (REST/GraphQL) și orice aplicație o consumă prin rețea.

Exemplu (contract REST):

```
POST /api/v1/validate-document
Content-Type: application/json

{ "cnp": "1960712123456" }

--> 200 OK
{ "isValid": true, "errors": [] }
```

Avantaje:

-  Scrii logica **o singură dată**, complet independent de platformă (RN, iOS, Android, web, orice).
-  Actualizezi logica **fără să republici** aplicațiile în store.
-  Logica sensibilă rămâne pe server (nu poate fi reverse-engineered din app).
-  Ideal pentru reguli de business ce se schimbă des.

Dezavantaje:

-  **Necesită internet** — nu merge offline.
-  Latență de rețea.
-  Nu e potrivit pentru lucruri strict on-device: criptare locală, senzori, NFC/eID, procesare de imagini în timp real.
-  Trebuie să întreții infrastructură (server, scaling, securitate).

5.4 Core partajat C++ / Rust

Pentru algoritmi grei, performanță maximă și portabilitate extremă, scrii nucleul în **C++** sau **Rust** și expui bindings per platformă (JNI pentru Android, C-interop/Swift pentru iOS). Abordare folosită de librării ca cele de la Google, Dropbox (Djinni), signal etc.

Avantaje:

- ✓ Performanță maximă, un singur nucleu de logică.
- ✓ Reutilizabil chiar și dincolo de mobil (desktop, embedded, server).

Dezavantaje:

- ✗ **Complexitate mare:** build cross-platform, toolchain, memory safety (la C++).
- ✗ Bindings și marshalling de date manuale, predispuse la erori.
- ✗ Necesită expertiză de nișă. Recomandat doar când chiar ai nevoie (procesare de imagini/semnal, criptografie custom).

6. Cum se leagă React Native de SDK

Aplicația ta React Native **nu e specială** — devine doar încă un consumator al SDK-ului, printr-un **Native Module** subțire care expune metodele SDK către JavaScript.

```
React Native (JS/UI)
  | apel JS
  ▼
Native Module (strat subțire, Kotlin + Swift)
  | apel nativ
  ▼
CORE SDK (KMP / nativ) ← aceeași logică ca în celelalte apps
```

Exemplu — Native Module (partea Android, Kotlin):

```
class DocumentModule(reactContext: ReactApplicationContext) :
    ReactContextBaseJavaModule(reactContext) {

    private val validator = DocumentValidator() // din core SDK

    override fun getName() = "DocumentModule"

    @ReactMethod
    fun validateCnp(cnp: String, promise: Promise) {
        val result = validator.validateCnp(cnp)
        val map = Arguments.createMap()
        map.putBoolean("isValid", result.isValid)
        promise.resolve(map)
    }
}
```

Apel din JavaScript (React Native):

```
import { NativeModules } from 'react-native';
const { DocumentModule } = NativeModules;

const result = await DocumentModule.validateCnp('1960712123456');
console.log(result.isValid);
```

Astfel, logica reală stă în SDK, iar RN doar o apelează — exact ca iOS-ul sau Android-ul nativ.

7. Tabel comparativ

Criteriu	KMP	Native x2	Backend/API	C++/Rust
Un singur cod pentru logică	✓ Da	✗ Nu	✓ Da	✓ Da
Rulează offline / on-device	✓ Da	✓ Da	✗ Nu	✓ Da
Efort de mentenanță	● Mic	● Mare	● Mic	● Mediu-mare
Cod native idiomatic	● Aproape	✓ Perfect	—	● Bindings
Actualizare fără republicare	✗ Nu	✗ Nu	✓ Da	✗ Nu
Curba de învățare	● Medie	● Mică	● Mică	● Mare
Potrivit pentru logică sensibilă/secretă	● Parțial	● Parțial	✓ Da	● Parțial
Integrabil în orice app Android + iOS	✓ Da	✓ Da	✓ Da (via rețea)	✓ Da

Regula de aur:

- Logica poate sta pe server → **Backend/API**.
- Logica trebuie on-device și vrei un singur cod → **KMP**.
- Vrei native pur și ai resurse pentru dublă mentenanță → **Native x2**.
- Algoritmi grei, performanță critică → **C++/Rust**.

8. Cum proiectezi API-ul public al SDK-ului

Un SDK bun are un **contract clar și stabil**. Recomandări:

- API minimal și explicit** — expune doar ce e necesar. Ascunde detaliile interne (`internal` / `private`).
- Fără dependențe de UI** — SDK-ul nu trebuie să știe de ecrane, culori, framework de UI.
- Tipuri clare de intrare/ieșire** — folosește modele/ `data class` , nu `Map` / `dictionary` amorse.
- Erori tratate explicit** — `Result` , tipuri de eroare dedicate, nu excepții „la nimereală”.
- Asincronie clară** — definește cum expui operațiile lungi (callbacks, `suspend` , `async/await` , Promise).

6. **Fără stare globală ascunsă** — preferă instanțe injectate, ușor de testat.

7. **Documentație + exemple** — fiecare metodă publică documentată.

Exemplu de contract (pseudo):

```
SmartIdSDK
├─ configure(apiKey, environment)    // inițializare
├─ DocumentValidator
│   └─ validateCnp(cnp) → ValidationResult
├─ SecureStorage
│   ├── save(key, value)
│   └─ load(key) → String?
└─ models
    ├── ValidationResult { isValid, errors }
    └─ Environment { PRODUCTION, STAGING }
```

9. Împachetare și distribuție

Platformă	Format artefact	Canal de distribuție
Android	<code>.aar</code> / Maven	Maven Central, GitHub Packages, repo Maven privat
iOS	<code>XCFramework</code> / Swift Package	Swift Package Manager (Git), CocoaPods, repo privat
KMP	ambele de mai sus	publicare simultană din Gradle

Bune practici:

- **Versionare semantică** (SemVer): `MAJOR.MINOR.PATCH`.
- **Binar precompilat** pentru consumatori (nu-i pui să compileze tot sursa).
- **Changelog** la fiecare release.
- **Pipeline CI/CD** care buildează și publică automat ambele artefacte dintr-un tag.

10. Versionare, compatibilitate și mentenanță

- **SemVer strict**: schimbări incompatibile → bump de `MAJOR`.
- **Deprecări treptate**: marchezi ce urmează să dispară, oferi alternativă, ștergi la următorul `MAJOR`.
- **Compatibilitate minimă**: declară versiunile minime (minSdk Android, iOS deployment target).
- **Nu strica API-ul public** fără preaviz — consumatorii depind de el.
- **Testează pe versiuni multiple** de OS.
- **Un singur owner** al contractului public (evită „fiecare adaugă ce vrea”).

11. Securitate

- **Nu pune secrete în SDK** (chei API, parole) — pot fi extrase prin reverse engineering al binarului mobil.
 - Logica cu adevărat sensibilă → ține-o pe **server**.
 - Folosește **storage securizat** al platformei (Keychain pe iOS, Keystore/EncryptedSharedPreferences pe Android).
 - **Validează intrările** primite de la consumatori.
 - Pentru date în tranzit → TLS + certificate pinning acolo unde se justifică.
 - Minimizați permisiunile cerute.
-

12. Checklist final

- ☐ Am stabilit dacă logica trebuie on-device sau poate sta pe server.
 - ☐ Am ales varianta (KMP / native x2 / API / C++/Rust) în funcție de răspuns.
 - ☐ Am definit un **API public** minimal, clar, fără dependențe de UI.
 - ☐ SDK-ul nu conține secrete și folosește storage securizat.
 - ☐ Am configurat împachetarea nativă (`.aar` + `XCFramework` /Swift Package).
 - ☐ Am versionare semantică + changelog + pipeline CI/CD.
 - ☐ Aplicația React Native consumă SDK-ul printr-un native module subțire.
 - ☐ Am teste centralizate în SDK.
 - ☐ Am documentație și exemple pentru consumatori.
-

Rezumat într-o frază

React Native e o alegere **pentru UI**. Dacă vrei **logică reutilizabilă în orice aplicație nativă**, acea logică trebuie să trăiască într-un **core SDK separat** (ideal **Kotlin Multiplatform** pentru on-device, sau un **backend/API** dacă poate sta pe server), iar aplicația React Native devine doar **unul dintre consumatori** aceluui SDK.